

openGauss 优化器调研中期报告

一、研究背景与总体目标

openGauss 继承 PostgreSQL 的基于代价（Cost-Based）的查询优化框架，在此基础上加入了更丰富的 Hint 机制、扩展统计与 AI 统计模型、以及向量化和 CodeGen 等执行加速技术。本次中期阶段，我们围绕以下核心问题展开调研：

1. **决策依据：** 优化器如何枚举所有可能的执行路线（不同连接顺序、不同扫描方式等）。
2. **成本计算：** 优化器对每条候选路线的代价是如何建模和计算的，关键参数和统计信息是什么。
3. **Hint / AI 干预：** Hint 和 AI 驱动统计信息如何在代价模型之上“插手”优化器决策，覆盖或修正默认的最小代价选择。

在此基础上，我们为下一阶段的实践工作设计了三个方向：

- 复杂 Hint 机制的源码验证；
- 大表 JOIN 代价模型的估算与实际执行时间对比；
- CodeGen 技术在 openGauss 中的性能收益分析。

二、优化器决策依据：搜索空间与执行路线枚举

2.1 逻辑查询树与等价重写

SQL 首先经过解析阶段，构造成内部的逻辑查询树（Query 结构），包括投影、选择、连接、聚合等运算。随后在重写与预处理阶段进行一系列等价变换，例如：

- 谓词下推（将过滤条件尽量推到接近数据源的地方）；
- 子查询拉平/半连接改写（将子查询改写为 JOIN 或半连接）；
- 视图展开、常量折叠、公共子表达式消除等。

这些变换决定了优化器后续要探索的**搜索空间的逻辑形态**。

2.2 基表访问路径枚举

对每个基表或子查询，优化器会构造一个 RelOptInfo 结构，记录该关系的多种访问路径（Path）。在 src/include/optimizer/cost.h 中可以看到典型路径对应的代价函数原型：

```
extern void cost_seqscan(Path* path, PlannerInfo* root, RelOptInfo* baserel, ParamPathInfo* par
extern void cost_index(IndexPath* path, PlannerInfo* root, double loop_count);
extern void cost_bitmap_heap_scan(
    Path* path, PlannerInfo* root, RelOptInfo* baserel, ParamPathInfo* param_info,
    Path* bitmapqual, double loop_count);
extern void cost_cstorescan(Path* path, PlannerInfo* root, RelOptInfo* baserel);
```

含义示意：

- **顺序扫描 (Seq Scan)：** 全表扫描，每个页按 `seq_page_cost` 计价；
- **索引扫描 (Index Scan)：** 通过索引访问部分元组，同时考虑索引页与回表开销；
- **Bitmap Heap Scan：** 先以 bitmap 形式定位候选行，再批量访问数据页，有利于减少随机 I/O；
- **列存扫描 (cstorescan)：** 针对列存表，单行 CPU 成本相对更低。

同一个 `RelOptInfo` 下通常会存在多条 `Path`，每条路径都会被填充 `startup_cost` 和 `total_cost`，后续在 JOIN 构造时，基于这些路径进行组合。

2.3 连接顺序与连接算法枚举

对于多表连接，openGauss 使用“动态规划 + 启发式”策略枚举 JOIN 顺序：

1. 为每个基表构造初始 `RelOptInfo`；
2. 按照表集合的大小（1 表、2 表、3 表……）自底向上，枚举所有可能子集；
3. 对于每个子集，尝试将其拆分为 (outer, inner) 两部分，并基于不同 JOIN 算法生成候选 `Path`；
4. 为每个子集保留当前**最优（总成本最低）**的路径，避免重复计算。

在 `src/include/optimizer/cost.h` 中可以看到三种 JOIN 算法的代价入口：

```
extern void initial_cost_nestloop(PlannerInfo* root, JoinCostWorkspace* workspace,
    JoinType jointype, Path* outer_path, Path* inner_path,
    JoinPathExtraData* extra, int dop);

extern void initial_cost_hashjoin(PlannerInfo* root, JoinCostWorkspace* workspace,
    JoinType jointype, List* hashclauses, Path* outer_path, Path* inner_path,
    JoinPathExtraData* extra, int dop);

extern void initial_cost_mergejoin(PlannerInfo* root, JoinCostWorkspace* workspace,
    JoinType jointype, List* mergeclauses, Path* outer_path, Path* inner_path,
    List* outersortkeys, List* innersortkeys, JoinPathExtraData* extra);
```

含义：

- **Nest Loop Join：** 逐行扫描外表，对每行执行一次内表访问；

- **Hash Join**: 构建内表 hash 表, 对外表逐行探测;
- **Merge Join**: 依赖两边有序输入, 按排序顺序合并;

在 JOIN 路径生成时, 优化器会判断当前连接条件、索引、排序等是否满足三种 JOIN 算法的可行性条件, 如果可行则分别调用对应的 `initial_cost_*` 函数计算代价, 再在局部范围内选择较优方案。

2.4 搜索空间剪枝

为了防止搜索空间指数级膨胀, 优化器采用多种剪枝策略:

- 对相同输出的多个 Path 只保留总代价最低者;
- 在外连接、半连接等存在语义顺序约束的情况下, 跳过非法的 JOIN 顺序;
- 当某个子计划的代价远高于已有候选时提前淘汰, 不再基于其扩展更大子集;
- 在一些复杂场景可启用遗传算法 GEQO, 对 JOIN 顺序进行近似搜索而非全量枚举。

三、成本计算：数学模型、参数与统计信息

3.1 基础成本参数

在 `src/include/optimizer/cost.h` 中定义了一系列默认代价参数:

```
#define DEFAULT_SEQ_PAGE_COST      1.0
#define DEFAULT_RANDOM_PAGE_COST  4.0
#define DEFAULT_CPU_TUPLE_COST    0.01
#define DEFAULT_CPU_INDEX_TUPLE_COST 0.005
#define DEFAULT_CPU_OPERATOR_COST  0.0025
#define DEFAULT_CPU_HASH_COST      0.02
#define DEFAULT_EFFECTIVE_CACHE_SIZE 16384 /* measured in pages */
```

这些参数对应:

- I/O 成本: 顺序页 `seq_page_cost`、随机页 `random_page_cost`;
- CPU 成本: 处理一行的 `cpu_tuple_cost`、处理一次索引元组的 `cpu_index_tuple_cost`、执行一个算子/哈希运算的成本等;
- 内存与缓存: `effective_cache_size` 影响 I/O 命中率预估。

抽象地, 可以把总代价写成:

$$\begin{aligned}\text{Cost}(\text{total}) &= \text{Cost}(\text{startup}) + \text{Cost}(\text{run}) \\ \text{Cost}(\text{run}) &\approx \text{IO_cost} + \text{CPU_cost} + \text{Memory_penalty}\end{aligned}$$

3.2 典型算子的代价公式（示意）

1. 顺序扫描（Seq Scan）

\$\$

$\text{Cost}(\text{seq}) \approx \text{seq_page_cost} \times \text{pages}$

- $\text{cpu_tuple_cost} \times \text{rows}$

\$\$

2. 索引扫描（Index Scan）

\$\$

$\text{Cost}(\text{index}) \approx \text{index_page_cost} \times \text{index_pages_visited}$

- $\text{cpu_index_tuple_cost} \times \text{index_tuples}$
- $\text{cpu_tuple_cost} \times \text{rows}$

\$\$

3. Hash Join（简化）

\$\$

$\text{Cost}(\text{hash}) \approx \text{build_cost}(\text{inner})$

- $\text{probe_cost}(\text{outer})$
- hash_overhead

\$\$

4. Nested Loop（简化）

\$\$

$\text{Cost}(\text{nl}) \approx \text{Cost}(\text{outer})$

- $\text{rows}(\text{outer}) \times \text{Cost}(\text{inner_per_row})$

\$\$

这些公式在实现层面会结合更多细节，例如是否可以利用缓存、是否有索引、是否需要排序等。

3.3 统计信息与选择度估计

- openGauss 利用系统表 `pg_statistic`、`pg_statistic_ext` 存储列及多列统计信息，包括：
 - 行数、页数、空值比例；
 - 直方图（histogram）、最常见值（MCV, Most Common Values）；
 - 多列联合分布统计（扩展统计）。
- 在选择度计算方面，`clauselist_selectivity` / `clause_selectivity` 会结合这些统计信息计算谓词过滤比例，并在多个谓词组合时做一定的独立性假设修正。

在 `cost.h` 底部，有一个用于修正概率或代价的 Logistic 函数宏：

```
#define LOGISTIC_FUNC(variable, threshold, max_func_value, min_func_value, slope_factor) \
    ((variable > threshold && variable > 0) ? max_func_value : \
     2 * (max_func_value - min_func_value) / (1 + exp(-1 * slope_factor * variable)) \
     - (max_func_value - 2 * min_func_value))
```

这类非线性函数用于在统计极端或数据分布偏斜时进行平滑过渡，避免估算结果出现极端值，提升代价模型的鲁棒性。

四、Hint 机制：从解析到规划的源码路径

Hint 是用户对优化器的“硬性建议”或“软性引导”，可以在注释中写出，例如：

```
SELECT /*+ NESTLOOP(t1 t2) */
      t1.*, t2.*
FROM   big_table t1
JOIN   big_table2 t2 ON t1.key = t2.key
WHERE  t1.col > 100;
```

该 Hint 要求优化器对表 `t1` 与 `t2` 之间使用 Nested Loop Join，即便在无 Hint 情况下，优化器可能更偏向使用 Hash Join。

4.1 Hint 关键字解析（`parse_hint.cpp`）

在 `src/common/backend/parser/parse_hint.cpp` 中，定义了所有支持的 Hint 关键字数组：

```

#define HINT_NUM 17
#define HINT_KEYWORD_NUM 23

const char* G_HINT_KEYWORD[HINT_KEYWORD_NUM] = {
    (char*) HINT_NESTLOOP,
    (char*) HINT_MERGEJOIN,
    (char*) HINT_HASHJOIN,
    (char*) HINT_LEADING,
    (char*) HINT_ROWS,
    (char*) HINT_BROADCAST,
    (char*) HINT_REDISTRIBUTE,
    (char*) HINT_BLOCKNAME,
    (char*) HINT_TABLESCAN,
    (char*) HINT_INDEXSCAN,
    (char*) HINT_INDEXONLYSCAN,
    (char*) HINT_SKEW,
    (char*) HINT_PRED_PUSH,
    (char*) HINT_PRED_PUSH_SAME_LEVEL,
    (char*) HINT_REWRITE,
    (char*) HINT_GATHER,
    (char*) HINT_NO_EXPAND,
    (char*) HINT_SET,
    (char*) HINT_CPLAN,
    (char*) HINT_GPLAN,
    (char*) HINT_SQL_IGNORE,
    (char*) HINT_CHOOSE_ADAPTIVE_GPLAN,
    (char*) HINT_NO_GPC,
};

```

KeywordDesc 函数用于将内部枚举值映射到对应的字符串：

```

static const char* KeywordDesc(HintKeyword keyword)
{
    const char* value = NULL;
    /* In case new tag is added within the old range. Keep the LFS as the newest keyword */
    Assert(HINT_KEYWORD_NO_GPC == HINT_KEYWORD_NUM - 1);
    if ((int)keyword >= HINT_KEYWORD_NUM || (int)keyword < 0) {
        elog(WARNING, "unrecognized keyword %d", (int)keyword);
    } else {
        value = G_HINT_KEYWORD[(int)keyword];
    }

    return value;
}

```

解析流程大致为：

1. 从 SQL 注释中提取 Hint 字符串，比如 `/*+ NESTLOOP(t1 t2) */`；
2. 调用 `hint_gram`（在 `parser/gramparse.h` 中声明的专用语法分析器）将 Hint 文本解析为内部的 Hint AST；
3. 将解析结果保存到 `HintState` 结构中，挂接到 `PlannerInfo` 或查询树上。

4.2 Hint 对规划阶段的影响

进入规划器后，`PlannerInfo` 会持有一个 `HintState` 指针供后续使用。在 JOIN 和 SCAN 路径枚举时，相关代码会主动检查 Hint 是否存在，例如：

- 对某个基表，如果有 `TABLESCAN(t1)` Hint，则可能禁止生成 `IndexPath`、`BitmapHeapPath`，只保留 `SeqScanPath`；
- 对两表 JOIN，如果有 `NESTLOOP(t1 t2)` Hint，则：
 - 优先生成并保留 `NestPath`；
 - 可能不再生成 `HashPath` 或将其标记为不可选；

关键点： Hint 的生效方式通常是在“路径生成阶段”就**约束或过滤候选路径集**，而不是在所有计划生成完毕后“强行替换”计划。这意味着：

- 优化器的代价计算仍然发生，只是发生在一个被 Hint 限制后的候选集合中；
- 若 Hint 选择了一个本来代价更高的 Path，优化器会照单全收，导致计划性能可能变差。

4.3 源码验证实验思路

下一阶段，可以设计如下 Hint 源码验证实验：

1. 选取一条多表 JOIN 查询，在无 Hint 情况下观察优化器倾向选择 Hash Join；
2. 加入 `NESTLOOP` Hint，强制其使用 Nested Loop；
3. 在 `parse_hint.cpp` 中 Hint 解析函数处设置断点，确认 Hint 正确解析为内部结构；
4. 在 JOIN Path 构造代码处（如 `make_join_rel` 相关实现）设置断点，检查：
 - 在有/无 Hint 场景下，哪些 `JoinPath` 被构造或被跳过；
 - 最终被选中的 `JoinPath` 类型是否发生变化。

通过对比 `EXPLAIN` 输出来验证最终物理计划的差异，形成“Hint 覆盖默认最小代价决策”的完整链路说明。

五、大表 JOIN：Hash Join 与 Nested Loop 代价模型对比

这一部分是后续实验的重点：通过调试器与 `EXPLAIN (ANALYZE)`，对比优化器对大表 JOIN 的代价估算与实际执行时间。

5.1 实验数据与 SQL 设计

示例：

```
-- 1. 创建大表
CREATE TABLE big_a(id int, key int, val text);
CREATE TABLE big_b(id int, key int, val text);

-- 2. 创建索引（便于 Nested Loop 使用 Index Scan）
CREATE INDEX idx_big_a_key ON big_a(key);
CREATE INDEX idx_big_b_key ON big_b(key);

-- 3. 无 Hint 的 JOIN 查询
EXPLAIN (ANALYZE, BUFFERS)
SELECT a.*, b.*
FROM   big_a a
JOIN   big_b b ON a.key = b.key
WHERE  a.id < 100000;

-- 4. 强制使用 Nested Loop 的 Hint 版本
EXPLAIN (ANALYZE, BUFFERS)
SELECT /*+ NESTLOOP(a b) */
      a.*, b.*
FROM   big_a a
JOIN   big_b b ON a.key = b.key
WHERE  a.id < 100000;
```

期望：

- 无 Hint 场景：若数据量较大且为等值连接，优化器通常会偏向 Hash Join；
- 有 `NESTLOOP(a b)` Hint：即使代价模型更偏向 Hash Join，计划也会强制转为 Nested Loop。

5.2 在优化器中观察 Hash/NL 的原始代价

在源码中，Hash Join 与 Nested Loop 的代价计算入口在 `cost.h` 中声明，具体实现位于 `costsize.c`：

```
extern void initial_cost_hashjoin(PlannerInfo* root, JoinCostWorkspace* workspace,
    JoinType jointype, List* hashclauses, Path* outer_path, Path* inner_path,
    JoinPathExtraData* extra, int dop);

extern void initial_cost_nestloop(PlannerInfo* root, JoinCostWorkspace* workspace,
    JoinType jointype, Path* outer_path, Path* inner_path,
    JoinPathExtraData* extra, int dop);
```

调试步骤示意（Linux + gdb, Windows 可使用 VS 或 CLion 等 IDE 调试）：

```
(gdb) break initial_cost_hashjoin
(gdb) break initial_cost_nestloop
(gdb) run
# 在执行测试 SQL 时命中断点
(gdb) print *outer_path
(gdb) print *inner_path
(gdb) print *workspace # 查看 startup_cost、total_cost 等
```

在无 Hint 与有 Hint 两种执行下记录：

- Hash Join 的 startup_cost 、 total_cost ；
- Nested Loop 的 startup_cost 、 total_cost ；
- outer_path->rows 、 inner_path->rows 等行数估计。

5.3 与实际执行时间对比

使用 EXPLAIN (ANALYZE, BUFFERS) ，收集两种场景下的实际执行数据：

- 每个 JOIN 节点的 Rows （估算行数）与 Actual Rows （实际行数）；
- Total Cost （优化器估算）与 Actual Total Time （实际耗时）；
- 缓存命中情况（ Buffers 信息）。

可以整理成表格对比：

场景	优化器选用 JOIN	估算总成本	实际耗时 (ms)	估算行数 vs 实际行数
无 Hint	Hash Join	10000	120	较接近 / 偏差
Hint: NESTLOOP(a b)	Nested Loop	30000	600	较接近 / 偏差

若优化器估算的代价排序与实际耗时排序大体一致，则说明代价模型在此场景下比较可靠；若偏差较大，可以进一步分析：

- 统计信息是否准确（表、索引是否最近 ANALYZE）；
- 数据分布是否高度倾斜、是否存在相关列被当作独立处理；
- 是否可以通过扩展统计或 AI 统计模型改善估算。

六、AI 驱动统计信息：BayesNet 模型的介入

为提升复杂场景下的选择度估计精度，openGauss 引入了 AI 统计模块，主要体现在 `src/include/optimizer/aioptimizer.h` 中。

6.1 核心数据结构与接口

`StatsModel` 类用于封装基于 BayesNet 的统计模型：

```
class StatsModel {
public:
    void *bayesNetModel;
    uint64_t processedtuples;
    MemoryContext mcontext;

    bool create_model(const char *model_name, HyperparameterValues *hyper,
                     TableScannerSample2 *reader);
    double predict_model(Datum *values, Oid *typids, bool *isnulls,
                        int ncolumns, bool *iseliminate);
    bool load_model(const char *model_name);
    bool search_model_from_cache(const char *model_name);
    // ...
};
```

相关辅助结构包括：

- `HyperparameterValues`：描述模型的结构规模（节点数、边数、MCV 数量等）；
- `SampleDataSet` 与 `TableScannerSample2`：封装采样数据集，提供 `fetch()` / `rescan()` 接口，供模型训练时迭代访问样本。

`TableScannerSample2::fetch()` 示例：

```

TupleScan *fetch()
{
    if (loc >= numrows) {
        return nullptr;
    }
    for (int i = 0; i < datumTuple.nattrs; i++) {
        int64 index = loc + i * this->numrows;
        datumTuple.values[i] = values[index];
        datumTuple.isnull[i] = isnull[index];
    }
    if ((++datumTuple.cnt) >= freqs[loc]) {
        datumTuple.cnt = 0;
        loc++;
    }
    return &datumTuple;
}

```

这段代码展示了如何从采样数据中逐条取出多列值，用于训练或预测。

6.2 与传统统计的协同

传统统计信息（直方图、MCV、单列/少量多列统计）在以下情况下容易出问题：

- 复杂多列联合谓词（如 `WHERE a = 1 AND b = 2 AND c BETWEEN ...`）；
- 多表多列 JOIN，字段间存在强相关性；
- 数据高度倾斜、热点值集中等。

AI 模块通过 BayesNet 建模多列联合分布，`predict_model` 可以给出更准确的：

- 多列谓词的联合选择度 $P(\text{condition})$ ；
- 多列 JOIN 条件下的输出行数估计。

在代价计算阶段，如果启用了 AI 统计，优化器会在传统统计的基础上调用 `predict_model` 得到更精细的选择度估计，从而修正 `rows` 和 `total_cost` 的计算结果。

七、效率加速：CodeGen（CPLAN/GPLAN）与 Hint 入口

7.1 CodeGen 的动机

传统解释执行模式下，每处理一行数据、每执行一个表达式，都需要通过解释层进行函数调用和类型判断，会产生大量分支预测失败和函数调用开销，特别是在：

- 复杂 WHERE 条件（多层函数嵌套、复杂表达式）；
- 大量聚合或计算列；
- 多表 JOIN 且 JOIN 条件较复杂的场景。

CodeGen（JIT，即时编译）通过将关键表达式在运行时编译为机器码，显著减少解释层开销，提高 CPU 利用率。

7.2 Hint 中的 CPLAN/GPLAN

在 Hint 关键字数组中可以看到与 CodeGen / 计划形态相关的 Hint：

```
(const char*) HINT_CPLAN,  
(const char*) HINT/GPLAN,  
(const char*) HINT_CHOOSE_ADAPTIVE/GPLAN,
```

这些 Hint 用于控制：

- 是否生成“代码生成计划”（CPLAN）；
- 是否生成通用计划（GPLAN）或自适应计划（Adaptive GPLAN）。

具体实现细节分布在规划与执行层代码中（包括生成 IR、触发 JIT 编译、缓存编译结果等），但从调研角度可以归纳为：

- 对表达式密集型算子（过滤、投影、聚合、JOIN 条件）构建 IR；
- 在首次执行或达到某个“热度阈值”时触发 JIT 编译；
- 之后重复执行同一计划时直接复用机器码。

7.3 后续可行实验方向

在系统参数或 Hint 允许的情况下，可以设计以下实验：

1. 关闭 CodeGen：执行复杂表达式查询，记录 EXPLAIN (ANALYZE) 的耗时；
2. 打开 CodeGen 或添加相关 Hint：在相同数据和 SQL 下再次执行；
3. 对比不同表达式复杂度、不同行数规模下的性能提升比例；
4. 考察 CodeGen 编译时间与执行时间节省之间的平衡点，探索 CodeGen 的适用场景边界。

八、阶段总结与后续工作计划

8.1 阶段性结论

- 在优化器决策机制方面：
 - openGauss 通过“逻辑重写 + 动态规划枚举 + 代价比较”在庞大的搜索空间中寻找最优计划；
 - 对每个基表与 JOIN 子集构造多种 Path，并利用成本模型与统计信息进行筛选和剪枝；
 - Hash Join、Nested Loop、Merge Join 等 JOIN 算法都有清晰的代价模型和适用条件。
- 在代价模型与统计信息方面：
 - 代价模型将 I/O、CPU 与内存开销统一折算为“代价值”，通过参数（`seq_page_cost`、`cpu_tuple_cost` 等）进行调节；
 - 利用单列与扩展统计估计谓词选择度和 JOIN 输出行数，并通过 Logistic 等非线性函数对极端情况进行修正；
 - 引入 AI 统计（BayesNet）后，能够更准确地建模多列相关性和倾斜，提高估算精度。
- 在 Hint 与 CodeGen 方面：
 - Hint 并非简单“硬覆盖最终计划”，而是通过约束候选路径空间、禁止或优先某些 Path 来影响代价比较过程；
 - 过度或不恰当使用 Hint 可能压制优化器对统计信息和代价模型的利用，导致计划变差；
 - CodeGen（CPLAN/GPLAN）为表达式密集场景提供了显著的 CPU 性能提升潜力。

8.2 下一步工作计划

1. 复杂 Hint 机制源码验证：

- 选取一条默认情况下使用 Hash Join 的大表 JOIN 查询；
- 加入强制 Nested Loop 的 Hint；
- 在 `parse_hint.cpp` 与 JOIN 路径构造代码处设置断点，跟踪 Hint 从解析到 Planner 的全路径；
- 对比有/无 Hint 的计划差异，验证 Hint 如何覆盖默认最小代价决策。

2. 大表 JOIN 代价模型实证对比：

- 构造大规模表及索引，执行 Hash Join 与 Nested Loop 两种计划；
- 使用调试器记录 `initial_cost_hashjoin` 与 `initial_cost_nestloop` 的原始代价估算值；
- 通过 EXPLAIN (ANALYZE, BUFFERS) 采集实际执行行数与耗时，评估代价模型的准确性与偏差来源。

3. CodeGen 效率加速评估：

- 在关闭/开启 CodeGen（或通过 Hint 控制 CPLAN/GPLAN）的情况下，执行同一批复杂表达式查询；
- 对比不同数据规模和表达式复杂度下的性能变化，分析 JIT 编译开销与执行收益的平衡点；
- 总结适合采用 CodeGen 的典型场景与不适用场景。

通过上述研究与实验，我们将进一步完成从“理论分析”到“源码级验证、实证评估”的闭环，为后续在 openGauss 上进行更复杂的优化器研究和实践打下坚实基础。